

Argumentul lui `yerror()` e un sir de caractere continand un mesaj de eroare (de obicei "syntax error"). In general, la detectarea unei erori de sintaxa programul afiseaza impreuna cu mesajul de eroare si numarul liniei din fisierul de intrare in care a aparut eroarea. Variabila intreaga externa `ychar` contine numarul token-ului look-ahead din momentul detectarii erorii si poate ajuta la debugging.

Variabila intreaga externa `ydebug` e setata in mod normal la valoarea zero. Daca ea are o alta valoare, analizorul produce si o descriere detaliata a actiunilor sale, inclusiv informatii despre simbolul de intrare citit, si despre actiunile efectuate.

## 2.2 Modul de lucru al analizorului sintactic

Analizorul generat de Yacc pe baza specificatiilor date este relativ simplu si consta dintr-un **automat stiva**. Spre deosebire de automatele folosite la analiza lexicala care sunt automate finite de stare, yacc foloseste automat stiva.

Stiva este necesara pentru a putea impinge si extrage de pe stiva stari. Starea curenta este intotdeauna cea din varful stivei. Acest lucru permite parser-ului sa poata trata si limbaje independente de context, nu doar limbaje bazate pe gramatici regulate. Analizorul e capabil sa citeasca si sa memoreze simbolul de intrare urmator, numit token look-ahead.

### 2.2.1 Automat stiva

Automatul are 4 actiuni : *shift*, *reduce*, *accept* si *error*. Initial masina e in starea 0 (zero), stiva contine starea 0 si nici un token "look-ahead" nu a fost citit.

- Pe baza starii curente, se decide daca trebuie citit inainte ("look-ahead") un token pentru a hotari actiunea de efectuat (shift sau reduce). Daca e nevoie de un nou token, e apelat `ylex()` pentru a-l obtine.
- Folosind starea curenta si eventual token-ul "look-ahead", analizorul decide actiunea sa urmatoare si o efectueaza. Aceasta poate determina punerea pe stiva a unor stari (push), preluarea de pe stiva a unor stari (pop), consumarea tokenului urmator sau doar examinarea inainte a tokenului urmator.
- actiunea de *shift* a unui token consuma acest token de la intrare. Pentru a exemplifica shift asupra unui *token*, sa presupunem ca intr-o stare avem actiunea

`IF shift, and go to state 34`

Atunci, daca token-ul urmator e IF, starea 34 devine stare curenta si e pusa pe varful stivei. Token-ul IF este consumat (sters) din intrare.

- actiunea *Reduce* e realizata cand partea dreapta a unei reguli gramaticale a fost identificata. Se inlocuieste partea dreapta cu partea stanga.
  - pentru a decide actiunea de reduce, e posibil sa fie necesar sa se consulte token "look-ahead" (dar nu e obligatoriu)
  - actiunea de reduce e asociata unei reguli gramaticale individuale
  - se scot de pe stiva atatea stari cate simboluri sunt in partea dreapta a regulii
  - exemplu: fie regula

$A : X Y X;$

1. Pentru a aplica *reduce*, se elimina de pe stiva primele 3 stari. Aceste stari au fost puse pe stiva cand s-a recunoscut X, Y si Z.
2. Dupa ce s-au eliminat (pop) aceste stari, varful stivei contine starea in care analizorul era cand a inceput sa proceseze regula (pt care s-a facut reduce acum).
3. Folosind aceasta stare si simbolul nonterminal A din stanga regulii, se realizeaza ceea ce ar fi *shift* a lui A, adica se impinge pe stiva o noua stare. In automatul construit de yacc, acest pas e marcat de *goto*. Pasul de goto nu consuma token de la intrare. De exemplu, starea din varful stivei dupa eliminarea starilor asociate lui X, Y, Z (consideram ca mai sus, ca exista regula A: X Y Z;) contine o descriere ca:

A go to state 20

Starea 20 e impinsa pe stiva si devine stare curenta. Analizorul se comporta apoi ca si cand ar fi vazut partea stanga in acel moment.

- actiunea de *accept* indica faptul ca intreaga intrare a fost vazuta si ca aceasta se potriveste cu specificatia. Aceasta actiune apare cand token-ul deja citit din intrare este end-marker-ul, si indica faptul ca analizorul a terminat procesarea cu succes.
- actiunea *error* reprezinta un punct din care nu se mai poate continua analiza conform specificatiei. Tokenii deja consumati din intrare impreuna cu token-ul urmator din intrare nu pot fi urmati de nimic ce ar constitui o intrare corecta.

Analizorul raporteaza o eroare si incearca sa refaca situatia si sa reia analiza.

Actiunea de *reduce* e importanta si in tratarea actiunilor si valorilor furnizate de utilizator. Cand o regula este redusa, **codul furnizat cu regula e executat inainte de modificarea stivei**. In paralel cu stiva ce pastreaza starile, exista o alta stiva ce pastreaza valorile returnate de analizorul lexical si de actiuni. Cand are loc un shift, variabila externa *yyval* e incarcata (push) pe stiva de valori. Dupa revenirea din codul utilizator, e efectuata actiune de *reduce*. Cand e realizata actiunea goto, variabila externa *yyval* e incarcata (push) pe stiva valorilor. Pseudo-variabilele \$1, \$2, ... refera de fapt stiva de valori.

Intuitia e urmatoarea: "o actiune de tip shift presupune inaintarea in lista de tokeni, iar o actiune reduce marcheaza gasirea unei potriviri cu partea dreapta a unei productii".

Fiecare stare a parser-ului va coincide cu una sau mai multe situatii (situatie = regula gramaticala aflata in stadiu diferit de procesare a partii drepte). Cu punct se identifica pozitia pana la care o regula a fost procesata. De exemplu, avand regula  $A \rightarrow a b c$ , unde a si b au fost procesate, dar nu si c, vom scrie:

$A \rightarrow a b . c$

Fiecare stare a parser-ului va avea asociate astfel de reguli gramaticale cu punct. Primul pas pentru obtinerea automatului generat pentru parser este utilizarea optiunii yacc -v, ce produce fisierul *y.output* continand informatii despre stari.

## 2.3 Exemplul 1

Sa consideram specificatia de mai jos care, la compilare cu -v, produce fisierul *y.output* din Listing 1.

```
1  %token      DING DONG DELL
2
3  %%
4
5  rhyme      :   sound place;
6  sound      :   DING DONG;
7  place      :   DELL;
```

Figura 1: Specificatia fisierului Yacc

```
State 0
  0 $accept: . rhyme $end
    DING  shift, and go to state 1
    rhyme go to state 2
    sound go to state 3

State 1
  2 sound: DING . DONG
    DONG  shift, and go to state 4

State 2
  0 $accept: rhyme . $end
    $end  shift, and go to state 5

State 3
  1 rhyme: sound . place
    DELL  shift, and go to state 6
    place go to state 7

State 4
  2 sound: DING DONG .
    $default reduce using rule 2 (sound)

State 5
  0 $accept: rhyme $end .
    $default accept

State 6
  3 place: DELL .
    $default reduce using rule 3 (place)

State 7
  1 rhyme: sound place .
    $default reduce using rule 1 (rhyme)
```

Listing 1: Fisierul y.output pentru gramatica DING DONG DELL

| Stiva | Intrare              | Actiune realizata          | Stare curenta<br>(urmariti deplasarea .) |
|-------|----------------------|----------------------------|------------------------------------------|
| #0    | DING DONG DELL \$end | shift                      | \$accept: . rhyme \$end                  |
| #01   | DONG DELL \$end      | shift                      | sound : DING . DONG                      |
| #014  | DELL \$end           | reduce cu regula 2 (sound) | sound: DING DONG .                       |
| #03   | DELL \$end           | shift                      | rhyme: sound . place                     |
| #036  | \$end                | reduce cu regula 3 (place) | place: DELL .                            |
| #037  | \$end                | reduce cu regula 1 (rhyme) | rhyme : sound place .                    |
| #02   | \$end                | shift                      | \$accept: rhyme . <i>end</i>             |
| #025  |                      | accept                     | \$accept: rhyme <i>end</i> .             |
| #02   |                      |                            |                                          |

Tabela 1: Modul de functionare a automatului pe DING DONG DELL

Sa presupunem ca intrarea este: DING DONG DELL. Operatiile pe stiva decurg conform urmatorilor pasi, cu mentiunea ca punctul e utilizat pentru a indica ce a fost vazut si ce trebuie sa urmeze, in fiecare regula (vezi tabelul 1):

1. Initial, starea curenta e 0. Analizorul trebuie sa analizeze intrarea pentru a alege una dintre actiunile disponibile in starea 0, deci primul token, DING, e citit, devenind token look-ahead. In starea 0, actiunea pentru DING e shift, and go to state 1, deci starea 1 este pusa pe stiva si token-ul look-ahead e sters. Starea 1 devine stare curenta.
2. In starea 1, urmatorul token, DONG e citit, devenind token look-ahead. Actiunea pentru token-ul DONG este shift, and go to state 4, deci starea 4 e pusa pe stiva si token-ul look-ahead e sters. Stiva contine acum 0, 1 si 4.
3. In starea 4, fara a fi necesara consultarea look-ahead, analizorul reduce prin regula 2: sound : DING DONG. Aceasta regula are doua simboluri in partea dreapta, deci doua stari, 4 si 1, sunt luate de pe stiva, lasand in varful stivei starea 0.
4. In starea 0, consultand descrierea si cautand un go to pentru sound, se obtine: sound go to state 3. Astfel starea 3 e pusa pe stiva, devenind stare curenta.
5. In starea 3 e citit urmatorul token, DELL. Actiunea e shift, and go to state 6, deci starea 6 e pusa pe stiva, care contine acum 0, 3 si 6 si token-ul look-ahead e sters.
6. In starea 6, singura actiune e reducerea prin regula 3 (place). Aceasta are un simbol in partea dreapta, deci se ia o stare de pe stiva, 6, ramanand starea 3 in varful stivei.
7. In starea 3, actiunea go to pentru place e starea 7. Acum, stiva contine 0, 3 si 7.
8. In starea 7 singura actiune posibila e reducerea prin regula 1 (rhyme). Exista doua simboluri in dreapta, deci sunt preluate de pe stiva doua stari, lasand starea 0 din nou pe varful stivei.
9. In starea 0, exista un go to pentru rhyme, prin care analizorul intra in starea 2.
10. In starea 2 urmatorul token, end-marker, e citit (indicat prin '\$end' in fisierul y.output). Actiunea in starea 2 cand se recunoaste end-marker-ul e shift spre starea 5. Acum, stiva contine starile 0, 2 si 5.
11. In starea 5, actiunea implicita este accept, se sterge tokenul look-ahead, iar analiza este terminata cu succes.